Final Project Report – Fall 2025

1. Introduction

This project presents a relational database designed to support the operational and analytical needs of the Loyola University New Orleans Swimming Team. The database provides a structured system for storing swimmers' information, competition results, teams, injuries, and their personal best times.

The system is intended to help coaches and staff make data-driven decisions such as roster planning, event specialization, recruitment, identifying athletes who may graduate soon, analyzing performance trends, and managing meet participation.

Although the database is not a complete application, it forms a strong foundation that can be expanded with additional business logic, automation, or integration with a frontend system. The SQL scripts included in the repository allow users to recreate the entire database from scratch and execute operations that reflect realistic team workflows.

2. Installation Instructions

2.1 Requirements

To install and use the database, you need:

- PostgreSQL 15 or later
- A SQL client such as DataGrip, DBeaver, pgAdmin, or psql

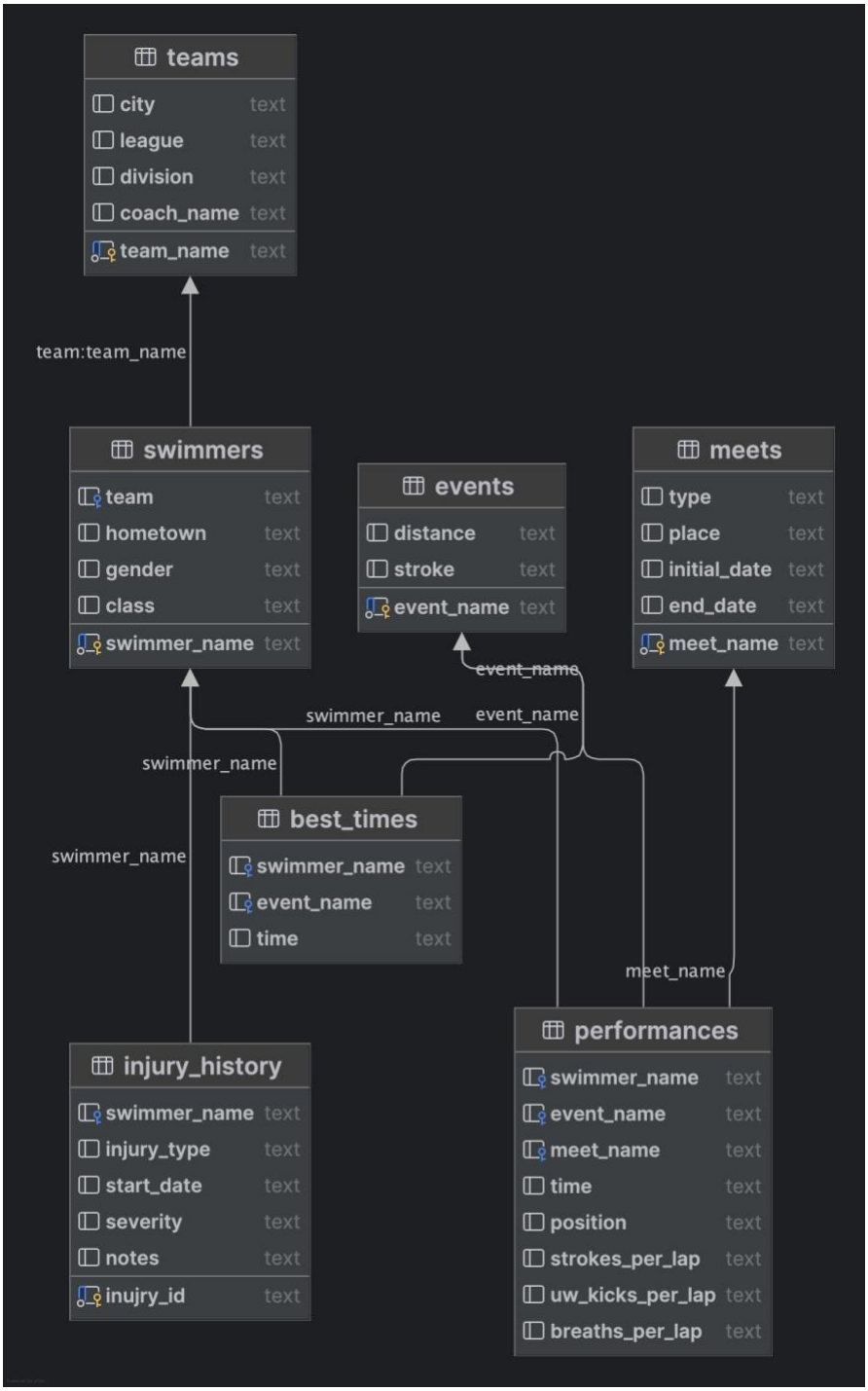
2.2 Setting Up the Database

1. Open your SQL client and connect to your PostgreSQL server.
2. Locate the folder SQL-DB-Creation-Scripts in your project.
3. Open the file script_order.txt — this file lists all SQL scripts in the correct execution sequence.
4. Run the scripts in the exact order shown. The scripts will:

- Create all tables
- Create foreign keys
- Load sample data

3. Data Model

3.1 Entity-Relationship Diagram



3.2 Data Model Overview

The model tracks swimmers, their teams, events, meet participation, injuries, and their personal best performances.

Key business relationships:

- One team has many swimmers
- One swimmer participates in many performances
- One event is linked to many performances
- Each meet has many performances
- Each swimmer has multiple injury records
- The best_times table stores each swimmer's fastest time per event

4. Table Descriptions

Entity 1: Swimmer

- Description: Represents an individual swimmer on a university team. Every swimmer is detailed by name, year, gender, hometown and team

- Example:

Column	Data Type	Key	Description
SwimmerName	TEXT	PK	Full name of the swimmer
Year	TEXT		School year of the swimmer
Gender	TEXT		Gender (M/F)
Hometown	TEXT		City of origin
Team	TEXT	FK→Team(TeamName)	Swimmer's team

Entity 2: Event

- Description: Every event is defined by the Distance and Stroke

- Example:

Column	Data Type	Key	Description
EventName	TEXT	PK	Event identifier (e.g., 100 Free)
Distance	INTEGER		Distance in yards
Stroke	TEXT		Stroke type (Free, Fly, Back, Breast, IM)

Entity 3: Performance

- Description: The performance of one specific swimmer in one specific event. It has measurable data such as Time, StrokeCountPerLap, BreathsPerLap*, UWKicksPerLap and Position

- Example:

Column	Data Type	Key	Description
SwimmerName	TEXT	FK → Swimmer(SwimmerName)	Swimmer identifier
EventName	TEXT	FK → Event(EventName)	Event identifier
MeetName	TEXT	FK → Meet(MeetName)	Meet identifier
Time	VARCHAR(8)		Swim time (MM:SS.CC)
Position	INTEGER		Finishing position
StrokeCountPerLap	INTEGER		Number of strokes per lap
UWKicksPerLap	INTEGER		Number of underwater kicks per lap
BreathsPerLap	INTEGER		Number of breaths per lap (optional)
Primary Key: (SwimmerName, EventName, MeetName)			Composite key

Entity 4: Team

- Description: The team is defined by its name, that appears in the swimmer data, City, League, Division and Coach

- Example:

Column	Data Type	Key	Description
TeamName	TEXT	PK	Team identifier
City	TEXT		Team location
League	TEXT		League name
Division	TEXT		Competition division
CoachName	TEXT		Coach name

Entity 5: Meet

- Description: The entity Meet defines the type of meet (championships, nationals, duel meet...), the pool (place) and the date

- Example:

Column	Data Type	Key	Description
MeetName	TEXT	PK	Name of the meet
Type	TEXT		Type of meet (duel, championship, etc.)
Place	TEXT		Location of the pool
Date	DATE		Date of the meet

Entity 6: Injury

- Description: The entity Injury tracks the health problems that have been affecting the swimmers, being swimmer a foreign key.

- Example:

Column	Data Type	Key	Description
Injury_id	TEXT	PK	ID of the injury
Swimmer_name	TEXT	FK	Swimmer that has the injury
Injury_type	TEXT		The injury
Start_date	DATE		When the injury happened
Severity	TEXT		How bad/important the injury is
Notes	TEXT		Notes from the doctor

Entity 7: Best_times

- Description: The entity Best_Times is defined by the swimmer and event_name and storages a time for each one of them, helping to know the best times of each swimmer for each event.
- Example:

Column	Data Type	Key	Description
swimmer_name	TEXT	FK	Swimmer identifier
event_name	TEXT	FK	Event identifier
Time	VARCHAR(8)		Swim time (MM:SS.CC)

5. Business Queries

5.1 Seniors Graduating

- Purpose: This query lists all swimmers who are Seniors, meaning they will graduate at the end of the academic year. Coaches can use this information to plan next season's roster and identify recruitment needs.

```
select
    swimmer_name
from
    public.swimmers
where class = 'SR'
order by swimmer_name;
```

5.2 Best Times in specific Event and Gender

- Purpose: This query returns the best (fastest) times in the 50 Free event for the male swimmers. Coaches can use this information to identify the best lineups for the 200free relay

```
SELECT
    p.swimmer_name,
    s.gender,
    p.event_name,
```

```
    p.time
FROM performances p
JOIN swimmers s
ON p.swimmer_name = s.swimmer_name
WHERE p.event_name = '50 Free'
  AND s.gender = 'M'
ORDER BY p.time;
```

5.3 Most-Swum Events in Specific Meet

- Purpose: This report shows which events were swum the most during a specific competition. Coaches use this information to understand meet structure, participation levels, and event popularity within a single meet.
- Instructions for use: To run this report for a different meet, CHANGE the meet_name value in the WHERE clause.

```
SELECT
    event_name,
    COUNT(*) AS total_swims
FROM performances
WHERE meet_name = 'Loyola New Orleans Vs. Life University'
GROUP BY event_name
ORDER BY total_swims DESC;
```

6. Business Operations

6.1 Insert a new swimmer

- Purpose: Inserts a brand-new swimmer into the database use this when recruiting new athletes or adding freshmen to the roster
- Instructions: To reuse this script for a different swimmer, change the VALUES below:
  1) swimmer_name → replace with the new student's name
  2) class → FR, SO, JR or SR
  3) gender → M / F
  4) hometown → City or origin
  5) team → Must exist in the team table

```
INSERT INTO public.swimmers (swimmer_name, class, gender, hometown,
team)
VALUES ('Gafo, Pelayo', 'SO', 'M', 'Madrid, Spain', 'Loyola University
New Orleans');
```

6.2 Update best times when a faster performance occurs

- Purpose: Updates personal best times for all swimmers and all events. If a performance time is faster than the time stored in personal bests, the best_time is updated accordingly.
- Instructions: You do not need to modify anything. Run this script after adding new performance results. It will automatically update only the times that are faster than the current personal bests.

```
UPDATE public.best_times bt
SET time = p.time
FROM performances p
WHERE bt.swimmer_name = p.swimmer_name
  AND bt.event_name = p.event_name
  AND p.time < bt.time;
```

7. Storage Procedure

7.1 Insert performance and auto-add meet

- Purpose: Inserts a new performance. If the meet does not exist in the meets table, it automatically inserts it using all required columns.

```
CREATE OR REPLACE PROCEDURE add_performance_entry(
    p_swimmer_name TEXT,
    p_event_name   TEXT,
    p_time         VARCHAR(8),
    p_meet_name    TEXT,
    p_type         TEXT,
    p_place        TEXT,
    p_initial_date DATE,
    p_end_date     DATE
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM meets
        WHERE meet_name = p_meet_name
    ) THEN
        INSERT INTO meets (meet_name, type, place, initial_date,
end_date)
        VALUES (p_meet_name, p_type, p_place, p_initial_date,
p_end_date);
    END IF;

    INSERT INTO performances (swimmer_name, event_name, time,
meet_name)
    VALUES (p_swimmer_name, p_event_name, p_time, p_meet_name);

END;
```

```
$$;

CALL add_performance_entry(
    'Brunmeier, Shay',
    '100 Fly',
    '51.20',
    'Tulane Winter Invitational',
    'Invite Meet',
    'Reily Student Recreation Center, New Orleans',
    '2025-12-15',
    '2025-12-16'
);
```

8. Script Usage Instructions

All the scripts are stored in the GitHub repository in their own folder. And there is a file 'script_order.txt' that determines the order in which the files to create the databases must be run. Always run:

- Table creation scripts
- Data load scripts

Order for this database:

- teams.sql
- teams_rows.sql
- swimmers.sql
- swimmers_rows.sql
- injury_history.sql
- injury_history_rows.sql
- events.sql
- events_rows.sql
- meets.sql
- meets_rows.sql
- performances.sql
- performances_rows.sql
- best_times.sql
- best_times_rows.sql

9. Conclusion

This database provides a solid, normalized foundation for managing the Loyola University Swimming Team's competitive, organizational, and performance data. Although simplified for the scope of this course, the system demonstrates the core concepts of relational modeling, normalization, foreign key design, and SQL-driven business logic.

Future improvements could include automatic performance scoring, meet scheduling automation, dashboards for coaches, and authentication-controlled data access. The current design supports all essential operations and can easily be extended.